Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Deriving Abstract Machines and Compilers

Master's Thesis

by

Mahmoud Mohamed Mohamed Elsayed Hamido

April 14, 2026

RPTU University Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die von mir vorgelegte Arbeit mit dem Thema „Deriving Abstract Machines and Compilers“ selbstständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit — einschließlich Tabellen und Abbildungen —, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Kaiserslautern, den 14.4.2026

Mahmoud Mohamed Mohamed Elsayed Hamido

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Zusammenfassung

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Contents

1. Introduction

- Language Specifications
  - A specification is an informal description of a language's behavior (e.g, C++, the JLS, WASM spec)
  - It serves as the basis of an implementation.
  - A mismatch between spec and implementation is a bug.
  - Formal verification closes this gap: we *prove* the implementation conforms to the spec.
- Various forms of (Formal) Semantics of Programming Languages [Pie02]
  - Natural (Big-Step) Semantics: relates expressions directly the values they evaluate to.
  - Operational (Small-Step) Semantics: describes computation as a sequence of reduction steps.
  - Denotational Semantics: maps programs to mathematical objects (e.g. the domain of agda values, functions, naturals, etc.)
  - Abstract Machines
- Some forms of semantics are easier to define than others.
  - Big-step semantics is often the most easiest to define, only define "productive" rules.
  - Small-step semantics is more verbose (context and reduction rules) but exposes fine-grained control over evaluation order and intermediate states.
  - Denotational semantics is often implemented as an interpreter in a meta-language (e.g. Agda),
- Thesis: Derive an implementation *from* a specification.
  - General strategy: start with a big-step semantics, then derive an abstract machine (operational semantics).
  - The derivation can be made rigorous: a certified interpreter or compiler extracted from a proof
  - Case studies: Arithmetic, Arithmetic with exceptions, and simple functional languages (STLC, PCF).
- Corollary: study how different semantics are related (and can be derived).
  - e.g. big-step $\iff$ small-step equivalence proofs.

2. Related Work

[PH21] [HW06] [Hut23]

3. Background

3.1. Agda

- Agda is a dependently typed programming language and proof assistant.
- The type system is capable of encoding specifications directly as types.
- Undermining principle: The Curry-Howard Correspondence [Wad15] (Propositions as Types), every proposition in logic corresponds to a type in a programming language.
- Constructive (Intutionistic) logic: a proposition is true if we can construct a proof (value) of it.
  - Truth: P is true if there exists a value of type P. Trivially represented by the unit type $\top$.

    ```
    data ⊤ : Set where
      tt : ⊤
    ```

  - Falsehood: P is false if there are no values of type P. Canonically represented by the empty type $\bot$.

    ```
    data ⊥ : Set where
    ```

    Note the absence of constructors for $\bot$. If a value of type $\bot$ was obtained, any proposition could be proven (ex falso quodlibet).

    ```
    ex-falso : {P : Set} → ⊥ → P
    ex-falso ()
    ```

    Note the empty pattern match.
  - Implication: $P \implies Q$ corresponds to a function type $P \to Q$.
  - Negation: $\neg P$ corresponds to $P \to \bot$.

    ```
    ¬_ : Set → Set
    ¬ P = P → ⊥
    ```

  - Conjunction: $P \wedge Q$ corresponds to a product type $P \times Q$

    ```
    data _×_ (P Q : Set) : Set where
      _,_ : P → Q → P × Q
    ```

    Generalization of product type: dependent pair (Σ-types) correspond to existential quantification ($\exists$).

    ```
    data Σ (A : Set) (B : A → Set) : Set where
      _,_ : (x : A) → B x → Σ A B
    ```

```
∃ : {A : Set} → (A → Set) → Set
∃ {A = A} P = Σ A P
```

  - Disjunction: $P \vee Q$ corresponds to a disjoint union type $P \uplus Q$.

```
data _⊎_ (P Q : Set) : Set where
  inl : P → P ⊎ Q
  inr : Q → P ⊎ Q
```

- Brief example of a simple proof in Agda (e.g. commutativity of addition).

3.2. Formalizing Language Semantics

- Structural Operational Semantics (Small-step): Relation on two terms, which demonstrates how the term is executed in an individual step. Reflexive-Transitive closure of the relation demonstrates the complete execution sequence to the final value (if one exists).
- Natural Semantics (Big-step): Relation on a term and the value it evaluates to. The proof trees contain the evaluation of subterms.
- Denotational Semantics: Map terms to domain (e.g. functions, naturals, etc.), often implemented as an interpreter in a meta-language (e.g. Agda).
- Comparison
  - Big-step semantics is often the most easiest to define, only define "productive" rules.
  - Non-terminating computations are not representable in big-step semantics, while small-step semantics can represent them through infinite reduction sequences.
  - Small-step semantics is fine-grained, allows the specification of evaluation order and intermediate terms during evaluation, but is more verbose (context ξ and reduction β rules).
  - Appropriate domain for a denotational semantics is not always obvious, and the meta-language may have different properties (e.g. totality) than the object language. i.e, explicit Closure vs implicit closure semantics for STLC.

3.3. Deriving Abstract Machines

4. Arithmetic

4.1. Syntax and Types

We define a simple arithmetic language with natural numbers and booleans, addition, and conditionals:

```
data Type : Set where
    Nat : Type
    Bool : Type

⊨_ : Type → Set
⊨ Nat  = ℕ
⊨ Bool = 𝔹

data ⊢_ where
    `_              : ⊨ T → ⊢ T
    _⊞_             : ⊢ Nat → ⊢ Nat → ⊢ Nat
    test            : ⊢ Nat → ⊢ Bool
    if_then_else_   : ⊢ Bool → ⊢ T → ⊢ T → ⊢ T

test' : ℕ → 𝔹
test' zero    = true
test' (suc n) = false
```

Note that the syntax is intrinsically typed: only well-typed expressions can be represented, and that we cannot construct non-terminating expressions.

4.2. Big-Step Semantics

The big-step semantics relates expressions to their values:

```
data _⇓_ : ⊢ T → ⊨ T → Set where
    `_ : (v : ⊨ T) → (` v) ⇓ v
    _⊞_ : e₁ ⇓ n₁ → e₂ ⇓ n₂
        → (e₁ ⊞ e₂) ⇓ (n₁ + n₂)
    test : e ⇓ n
        → (test e) ⇓ (test' n)
    if-then : e ⇓ true → e₁ ⇓ v
        → (if e then e₁ else e₂) ⇓ v
    if-else : e ⇓ false → e₂ ⇓ v
        → (if e then e₁ else e₂) ⇓ v
```

A value which is either a natural number or a boolean is a normal form, and evaluates to itself. The operands of addition are evaluated recursively, and the results are added. The if-then-else expression evaluates the condition first, and then evaluates either the "then" or "else" branch based on the result of the condition.

Evaluation is *deterministic*: if an expression evaluates to two different values, they must be equal.

```
deterministic : ∀ {e : ⊢ T} {v₁ v₂ : ⊨ T}
    → e ⇓ v₁ → e ⇓ v₂ → v₁ ≡ v₂
```

This is proven by structural induction on the evaluation derivations.

Every expression evaluates to *some* value.

```
total : ∀ (e : ⊢ T) → ∃ (λ v → e ⇓ v)
```

4.3. Small-Step Semantics

The small-step semantics describes computation as a sequence of reduction steps. Context rules (ξ-rules) specify where reduction can occur in one of the subterms, while β-rules perform the actual computation:

```
data _⇾_ : ⊢ T → ⊢ T → Set where
    ξ-⊞₁ : e₁ ⇾ e₁' → (e₁ ⊞ e₂) ⇾ (e₁' ⊞ e₂)
    ξ-⊞₂ : e₂ ⇾ e₂' → (` n₁ ⊞ e₂) ⇾ (` n₁ ⊞ e₂')
    β-⊞ : (` n₁ ⊞ ` n₂) ⇾ ` (n₁ + n₂)
    ξ-test : e ⇾ e' → (test e) ⇾ (test e')
    β-test-zero : test (` 0) ⇾ ` true
    β-test-suc : test (` suc n) ⇾ ` false
    ξ-if : e ⇾ e' → (if e then e₁ else e₂) ⇾ (if e' then e₁ else e₂)
    β-if-then : (if ` true then e₁ else e₂) ⇾ e₁
    β-if-else : (if ` false then e₁ else e₂) ⇾ e₂
```

The reflexive-transitive closure $\to^*$.

```
data _⇾*_ : ⊢ T → ⊢ T → Set where
    base  : e ⇾* e
    step  : e₁ ⇾ e₂ → e₂ ⇾* e₃ → e₁ ⇾* e₃
```

4.4. Denotational Semantics

The denotational approach interprets expressions directly as values.

```
_⟦_⟧ : ⊨ T → ⊢ T → ⊨ T
δ ⟦ ` v ⟧                = v
δ ⟦ e₁ ⊞ e₂ ⟧           = δ ⟦ e₁ ⟧ + δ ⟦ e₂ ⟧
δ ⟦ test e ⟧             = test' (δ ⟦ e ⟧)
δ ⟦ if e then e₁ else e₂ ⟧ with δ ⟦ e ⟧
... | true  = δ ⟦ e₁ ⟧
... | false = δ ⟦ e₂ ⟧
```

4.5. Abstract Machine

We systematically derive an abstract machine from the big-step semantics. The machine uses a stack of frames to track evaluation contexts. For each construct in the language, we define corresponding frames that represent the which subterm is being evaluated (the hole) and what context remains to be evaluated after it.

```
data Hole : Set where
    ◌ : Hole

data Frame : Type → Type → Set where
    _else_   : ⊢ T → ⊢ T → Frame T Bool
    _⊞₀_     : Hole → ⊢ Nat → Frame Nat Nat
    _⊞₁_     : ⊨ Nat → Hole → Frame Nat Nat

data Stack (Answer : Type) : Type → Set where
    []   : Stack Answer Answer
    _::_  : Stack Answer T₁ → Frame T₁ T₂ → Stack Answer T₂
```

The representation on the stack keeps track of the type of the (sub-)terms being evaluated.

```
data State (Answer : Type) : Set where
    ⊢_↓_ : Stack Answer T → ⊨ T → State Answer
    ⊢_↑_ : Stack Answer T → ⊢ T → State Answer
```

The machine operates in two modes while carrying a stack of frames, the eval mode (↑) where it evaluates an expression, and returning mode (↓) where it returns a value.

The machine transitions between states and mimics how an interpreter would evaluate expressions.

```
data _⇒_ : State Answer → State Answer → Set where
  step-`   : ⊢ stack ↑ ` v ⇒ ⊢ stack ↓ v
  step-⊞₁  : ⊢ stack ↑ (e₁ ⊞ e₂) ⇒ ⊢ stack :: (◌ ⊞₀ e₂) ↑ e₁
  step-⊞₂  : ⊢ stack :: (◌ ⊞₀ e₂) ↓ v₁ ⇒ ⊢ stack :: (v₁ ⊞₁ ◌) ↑ e₂
  step-⊞₃  : ⊢ stack :: (v₁ ⊞₁ ◌) ↓ v₂ ⇒ ⊢ stack ↓ (v₁ + v₂)
  step-if  : ⊢ stack ↑ (if e then e₁ else e₂) ⇒
             ⊢ (stack :: (e₁ else e₂)) ↑ e
  step-then : ⊢ (stack :: (e₁ else e₂)) ↓ true ⇒ ⊢ stack ↑ e₁
  step-else : ⊢ (stack :: (e₁ else e₂)) ↓ false ⇒ ⊢ stack ↑ e₂
```

4.6. Equivalence of Semantics

To formalize the relationship between semantics, we prove equivalence theorems ensuring that all approaches are logically equivalent.

4.6.1. Big Step $\Longleftrightarrow$ Small Step

4.6.2. Big Step $\Longleftrightarrow$ Denotational

4.6.3. Deriving the Rest

5. Exceptions

We extend the arithmetic language with exceptions. Unlike the base language where every expression terminates, we now introduce the possibility of runtime errors. The fundamental change is that expressions may either evaluate to a value *or* raise an exception.

5.1. Syntax

We extend the expression language with exception handling constructs:

```
data Exception : Set where
    err : Exception

data ⊢_ where
    `_                : ⊨ T → ⊢ T
    _⊞_               : ⊢ Nat → ⊢ Nat → ⊢ Nat
    test              : ⊢ Nat → ⊢ Bool
    if_then_else_     : ⊢ Bool → ⊢ T → ⊢ T → ⊢ T
    -- Exception handling
    throw             : Exception → ⊢ T
    try_catch_        : ⊢ T → (Exception → ⊢ T) → ⊢ T
```

5.2. Big-Step Semantics

The big-step semantics is modified to account for both successful evaluation and exception propagation. We introduce a disjoint union: terms evaluate to either a value or an exception.

```
data Result (T : Type) : Set where
    return  : ⊨ T → Result T
    raise   : Exception → Result T

data _⇓ʳ_ : ⊢ T → Result T → Set where
    `_ : (v : ⊨ T) → (` v) ⇓ʳ return v

    _⊞_
      : e₁ ⇓ʳ r₁ → e₂ ⇓ʳ r₂
      → (e₁ ⊞ e₂) ⇓ʳ (liftA2 (+) r₁ r₂)

    throw-rule : throw ex ⇓ʳ raise ex
```

```
try-return
    : e ⇓ʳ return v
    → (try e catch h) ⇓ʳ return v

try-catch : e ⇓ʳ raise ex
    → h ex ⇓ʳ r
    → (try e catch h) ⇓ʳ r
```

Definitions for convenience:

- $e \Downarrow v := e \Downarrow^r \text{return}\, v$
- $e \Uparrow ex := e \Downarrow^r \text{raise}\, ex$

5.3. Small-Step Semantics

Small-step semantics extends reduction rules to handle exception propagation. When an exception is raised, it propagates outward through the computation context:

```
data _⇾_ : ⊢ T → ⊢ T → Set where
    ξ-⊞₁ : e₁ ⇾ e₁' → (e₁ ⊞ e₂) ⇾ (e₁' ⊞ e₂)
    ξ-⊞₂ : e₂ ⇾ e₂' → (` n₁ ⊞ e₂) ⇾ (` n₁ ⊞ e₂')
    β-⊞  : (` n₁ ⊞ ` n₂) ⇾ ` (n₁ + n₂)

    -- Exception propagation: throw bubbles up through each context
    β-⊞-exn₁ : (throw ex ⊞ e₂) ⇾ throw ex
    β-⊞-exn₂ : (` n₁ ⊞ throw ex) ⇾ throw ex

    -- Try-catch handling
    ξ-try  : e ⇾ e' → (try e catch h) ⇾ (try e' catch h)
    β-try-throw : (try (throw ex) catch h) ⇾ (h ex)
```

5.4. Denotational Semantics

[HW06]

5.5. Abstract Machine

The abstract machine extends the base machine to handle exceptions. The stack now includes exception handler frames, allowing the machine to pop frames and invoke handlers when exceptions occur:

```
data Frame : Type → Type → Set where
    -- Base frames (from before)
    _else_          : ⊢ T → ⊢ T → Frame T Bool
    _⊞₀_            : Hole → ⊢ Nat → Frame Nat Nat
    _⊞₁_            : ⊨ Nat → Hole → Frame Nat Nat
```

```
    -- Exception handling frame
    catch          : (Exception → ⊢ T) → Frame T T

data State (Answer : Type) : Set where
    ⊢_↓_  : Stack Answer T → ⊨ T → State Answer
    ⊢_↯_ : Stack Answer T → Exception → State Answer
    ⊢_↑_  : Stack Answer T → ⊢ T → State Answer
```

The machine operates in three modes: eval (↑), return (↓), and exception return (↯). When an exception occurs, the machine transitions to exception return mode and unwinds the stack looking for a catch frame.

6. The Simply-Typed Lambda Calculus

6.1. Syntax

De Bruijn indices replace named variables with natural numbers indicating binding depth.

6.2. Semantics

- Big-step semantics: evaluation under an environment mapping variables to values
- Values: closed λ-abstractions (and their closures in an environment-based semantics)
- Small-step semantics: β-reduction and ξ-congruence rules; progress and preservation theorems
- Denotational semantics: agda types as domains.
- Closure semantics: explicit closure representation of functions.
- Problems (Termination Pragma, which asserts STLC is total, which isn't obvious to Agda)
- Agda's termination checker requires all recursive calls to be on structurally smaller arguments
- Logical Relations

7. Conclusion

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

List of Figures

A. My Code

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

B. My Ideas

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliography

[Hut23] Graham Hutton. "Programming language semantics: It's easy as 1,2,3". In: *Journal of Functional Programming* 33 (2023). URL: https://api.semanticscholar.org/CorpusID:247581669.

[HW06] Graham Hutton and Joel Wright. "Calculating an Exceptional Machine". In: *Trends in Functional Programming volume 5*. Ed. by Hans-Wolfgang Loidl. Selected papers from the Fifth Symposium on Trends in Functional Programming, Munich, November 2004. Intellect, Feb. 2006.

[PH21] Mitchell Pickard and Graham Hutton. "Calculating Dependently-Typed Compilers". In: *Proceedings of the ACM on Programming Languages* 5.ICFP (Aug. 2021).

[Pie02] Benjamin C. Pierce. *Types and Programming Languages*. 1st. The MIT Press, 2002. ISBN: 0262162091.

[Wad15] Philip Wadler. "Propositions as types". In: *Commun. ACM* 58.12 (Nov. 2015), pp. 75–84. ISSN: 0001-0782. DOI: 10.1145/2699407. URL: https://doi.org/10.1145/2699407.